Air-HUIM: Exact High-Utility Itemset Mining Under a Hard Memory Budget on Edge Hardware

Arya Gupta
Department of Computer Science and Engineering
University of Engineering and Management
Kolkata, India
aryagupta.dev@gmail.com

Avirupa Malakar
Department of Computer Science and Engineering
University of Engineering and Management
Kolkata, India
avirupamalakar@gmail.com

Department of Computer Science and Engineering
University of Engineering and Management
Kolkata, India

***Abstract*—High-utility itemset mining (HUIM) finds sets of items whose combined value, such as profit, exceeds a threshold. Its algorithms are designed and evaluated on desktops and servers with 16–128 GB of memory, and the reference implementations report memory use but never bound it. We present Air-HUIM, a Rust library of 23 HUIM algorithms that run exactly under a user-set memory budget. One memory ledger accounts for every structure that grows with the data; each algorithm spills, recomputes or partitions its state when the budget is reached, and refuses up front when the budget is below a computed minimum. The unifying idea is that the location of search state never changes the result: state can be spilled, recomputed from its parents, avoided by partitioning the search, or relaxed to a looser valid bound, and memory pressure costs time, never answers. On a 4 GB Raspberry Pi 4, across 14 datasets and budgets from 24 MB to 2 GB, no Air-HUIM run was killed for memory, all 905 runs with a reference output were exact, and a kernel-enforced limit killed no Air-HUIM run. At the same budgets, the reference Java library SPMF ran out of memory in 229 of 588 runs. Air-HUIM mines the 47.1-million-basket dunnhumby LGSR dataset on the Pi within 1 and 2 GB, where SPMF fails in all 8 runs. Verifying every disagreement by a full scan also revealed three correctness bugs in SPMF.**

***Index Terms*—high-utility itemset mining, memory-bounded algorithms, edge computing, Raspberry Pi, Rust**

I. Introduction

High-utility itemset mining generalises frequent itemset mining: each item in a transaction carries a quantity and a unit value, and the task is to find every itemset whose total utility in the database reaches a threshold `min_util` [1], [2]. Retail analysis is the standard application: which products together bring in the most revenue.

Two decades of work have made HUIM algorithms much faster [3]–[6]. They have been evaluated almost exclusively on well-provisioned machines. The standard empirical comparison [7] ran SPMF [8], the reference implementation of most of these algorithms, on a 4.4 GHz desktop with 128 GB of RAM and a 128 GB Java heap. Memory is reported as an outcome of a run, never as a constraint on it. Parallel work targets 96-thread servers [9] and GPUs [10].

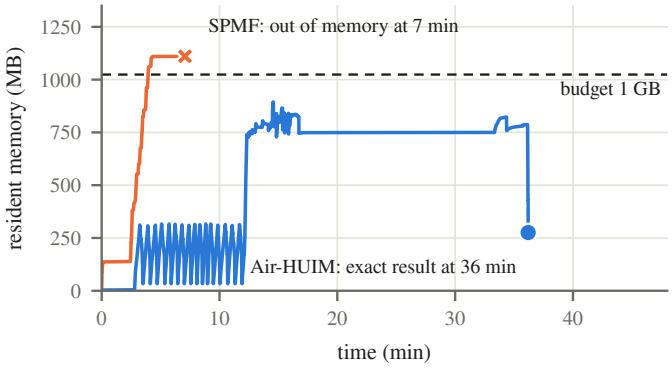


Fig. 1. Resident memory over time, FHM on 47.1 M baskets (LGSR) with a 1 GB budget on a 4 GB Raspberry Pi. SPMF grows until the JVM runs out of memory; Air-HUIM stays under the budget and returns the exact result. The oscillation before minute 12 is the external sort that builds the 1-itemset lists, spilling and refilling its buffer.

This leaves a practical question open: can these algorithms run *exactly* on the hardware where much retail and sensor data is produced, such as a single-board computer with a few gigabytes of RAM, without the analyst having to guess whether a run will fit? On a 4 GB Raspberry Pi, SPMF fails on 47.1 million baskets at every budget we tried, and on the classic benchmark datasets it runs out of memory at a 64 MB heap in 116 of 196 runs (Section VI). Fig. 1 shows the difference on one run.

We present **Air-HUIM**, a Rust library in which the memory budget is a first-class input. The user gives one number; every algorithm keeps the process within it, uses the memory it is given, and returns exactly the same itemsets at every budget. Our contributions are:

1) **Budgeted state, one idea for 23 algorithms** (Section III): four moves on search state (spill, recompute, partition, relax) that provably preserve the output, a resident core below which a run refuses, and an operator

matrix that gives every algorithm's memory model.
2) **An architecture that executes the moves** (Section IV): engines reserve every growing structure from one state manager (ledger, move policy, admission control, RSS feedback) that shares its budget with the buffer pool.
3) **An evaluation on a 4 GB Raspberry Pi** (Section VI): 1,503 runs over 14 datasets and budgets from 24 MB to 2 GB, against SPMF at the same budgets and under a kernel-enforced memory limit, plus a 47.1-million-basket retail dataset.
4) **Three correctness bugs in SPMF** (Section VII), found by checking every disagreement against a full scan of the data.

To our knowledge, Air-HUIM is the first HUIM library in Rust, and this is the first evaluation of exact HUIM on a single-board computer. Memory-aware HUIM itself is not new (Section II); what is new is a hard, user-set budget applied uniformly across a family of algorithms, with exactness verified at every budget.

II. Background and Related Work

Problem. A database D is a set of transactions; each transaction T assigns a quantity $q(i,T)$ to each of its items, and each item has a unit utility $p(i)$. The utility of itemset X in $T \supseteq X$ is $u(X,T) = \sum_{i \in X} p(i)\, q(i,T)$, and $u(X) = \sum_{T \supseteq X} u(X,T)$. Utility is not anti-monotone, so algorithms prune with upper bounds such as the transaction-weighted utility (TWU) [1], remaining utility [3] or sub-tree utility [5].

Algorithm families. *Two-phase* algorithms generate candidates with an over-estimate and verify them in a second scan [1]. *Tree* algorithms (IHUP, UP-Growth, UP-Growth+ [6]) compress the database into a prefix tree and also verify candidates. *Utility-list* algorithms (HUI-Miner [3], FHM [4], mHUIMiner [11]) join per-itemset lists and need no second phase. *Projection* algorithms (EFIM [5]) project and merge the database at each search step. Top-k variants (TKO, TKU [12]) raise the threshold during the search.

Libraries. SPMF (Java) [8] implements most published HUIM algorithms and is the de facto reference. PAMI (Python) [13] covers fewer. Neither bounds memory; the Java heap limit only decides when a run fails.

Parallel HUIM. DPHIM [9], [14] decomposes the search dynamically and places tasks NUMA-aware, reaching up to 72.7× over a tuned serial run and 40.4× at 96 threads. GA-HUIM maps HUIM onto GPUs [10]. These target throughput on large machines; memory is not bounded.

Memory-aware mining. Erwin et al. use a parallel projection that falls back to disk when memory is short [15]; Vu and Nguyen propose a disk-based HUIM algorithm [16]; PTM partitions by prefix to mine top-k HUIs on massive data [17]. Out-of-core frequent itemset mining is mature [18]. Each of these is one algorithm; none takes a user-set budget or covers a family of algorithms.

Mining on single-board computers. The closest prior work runs Apriori, a frequent (not utility) itemset algorithm, on a *cluster* of Raspberry Pis to save energy [19]. A search of OpenAlex, arXiv and Crossref found no HUIM evaluation on a single-board computer.

III. Budgeted State: One Idea for 23 Algorithms

HUIM algorithms differ in data structures, but every exact one has the same shape: it enumerates itemsets in a search tree, attaches *state* to the nodes, prunes with upper bounds computed from that state, and reports each enumerated itemset whose exact utility reaches `min_util`. The state falls into five kinds: the *input* database (after TWU filtering); *base* structures built from it (1-itemset utility lists, a prefix tree, a projected database); *derived* state of inner nodes (the list or projection of $P \cup \{x\}$, computed from its parents); *pruning aids* (the pair-utility matrix EUCS, local and sub-tree utilities); and *candidates* awaiting verification in two-phase and tree algorithms.

Air-HUIM rests on one observation: **where a piece of state lives does not affect which itemsets are reported, as long as the state is either restored exactly or replaced by something that is still a valid upper bound.** So when the budget runs out, each piece of state can be handled by one of four *moves* (Fig. 2), and below an irreducible *resident core* the run refuses:

1) **Spill:** write it to the page store; a reload restores it byte for byte.
2) **Recompute:** drop derived state and rebuild it from its parents, which are alive higher in the search; the rebuild is a deterministic function of them.
3) **Partition:** split the work so that each part's state fits, such that every itemset is enumerated in exactly one part, with access to all transactions that contain it.
4) **Relax:** drop or coarsen a pruning aid, but only towards a looser bound that is still an upper bound. Pruning less costs time, never results.

Refuse: if even the resident core (what one worker needs to make progress) does not fit, stop before mining and report the smallest workable budget.

Proposition (exactness under moves). *For an exact algorithm whose bounds are valid upper bounds, any sequence of the four moves leaves its output unchanged.*

Sketch. Spill and recompute restore identical state, so every later computation is identical. Partition enumerates each itemset once, from all transactions containing it, so its utility is unchanged. Relax keeps every bound valid, so no itemset with $u(X) \geq \texttt{min_util}$ (or ancestor of one) is pruned; extra enumerated itemsets are reported only if their exact utility reaches the threshold. In every case the reported set is exactly $\{X : u(X) \geq \texttt{min_util}\}$. □

The proposition also says what *must not* be done. A truncated EUCS that forgets pairs and treats missing pairs as zero is a *tighter*, invalid bound: it prunes valid itemsets. We found exactly this failure in an early version, which is why EUCS is disabled rather than truncated. The one deliberate exception is the metaheuristics, which are approximate by design: their set of already-reported itemsets becomes a Bloom filter, which can only skip a new itemset.

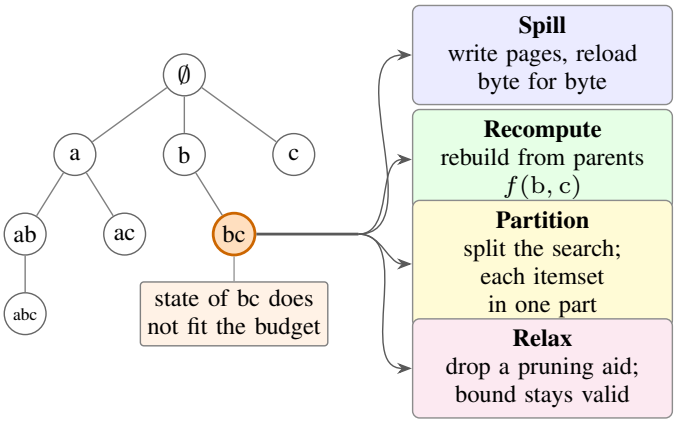


Fig. 2. The four moves. When a node's state does not fit, it can be spilled, recomputed from its parents, avoided by partitioning the search, or, for pruning aids only, relaxed to a looser valid bound. Each move leaves the set of reported itemsets unchanged.

The operator matrix. Table I applies the moves to each kind of state in each engine. Reading a row gives an algorithm's complete memory model: what is spilled, recomputed, partitioned or relaxed, and what stays resident. Admission control computes the resident core from the first pass, so the minimum budget is known before mining starts.

IV. Architecture

Fig. 3 shows how the moves are executed. Engines never allocate growing state directly: they ask the *state manager* for a reservation, and a refusal is their signal to apply a move. The state manager owns the ledger, the cost model that chooses between spilling and recomputing, admission control, and a feedback loop from the real RSS. Below it, one storage layer serves every engine.

The budget. The user passes one number B (e.g. `-b 64` for 64 MB). The goal is that the process RSS stays within B, that the run uses B (data stays in RAM while it fits), and that the output is identical at every B.

One ledger. A single `MemoryGuard` accounts for everything that grows with the data: native structures (utility lists, projections, trees, the pair-utility matrix, the top-k heap) through RAII reservations released when the structure is dropped; buffer-pool frames, which the pool charges when it caches a page; and the process baseline, read once from `VmRSS`. The ledger's limit is B minus a margin of 7 MB + 3% (at most 128 MB and at most $B/2$) for what it cannot see: allocator metadata, I/O buffers and thread stacks. A reservation either succeeds or tells the caller to fall back (spill, recompute or partition); small bounded buffers use a forced reservation that is accounted but never refused.

Buffer pool and spill. Spilled data lives in a page store on disk behind a buffer pool that shares the ledger. Cached pages are copies of spilled data, so when a native reservation fails the pool evicts first. Pages of dead search branches are discarded without being written. Small spilled lists are packed into shared pages so that a starved search does not create one file per list.

Admission control. After its first pass, each engine estimates the memory it cannot spill, as a fixed part plus a per-worker part. If one worker does not fit, the run stops at once with the smallest workable budget ("Use at least `-b 33`"); otherwise the thread count is lowered until the per-worker sets use at most half of the free budget.

RSS feedback. The ledger sees allocations, not resident pages: freed memory that the allocator keeps, and fragmentation between long- and short-lived blocks, raise RSS above the ledger. A background thread reads RSS every 250 ms, returns free heap pages to the OS (`malloc_trim`), and lowers the ledger limit by any excess over 92% of B (never below half its starting value), raising it again when RSS falls.

Dirty pages. Under a kernel memory limit (Linux cgroups), dirty page-cache pages count against the process. Spill writes therefore start write-back immediately and wait for it every 16 MB, so a burst of spilling cannot push the process over a hard limit.

Threads. The top-level search is split into tasks run by a work-stealing pool; results go to one writer thread over a bounded channel whose capacity is reserved from the ledger, so a slow disk applies backpressure instead of growing memory.

V. Applying the Moves to Each Engine

This section explains the non-obvious entries of Table I.

Utility-list engine. A list body longer than one chunk (4,096 entries, smaller under tiny budgets) is a sequence of chunks; a join reads the prefix list and the two extension lists one chunk at a time and writes the result chunk by chunk, so its working set is constant in the list lengths. When a new list does not fit, a cost model chooses between writing it to disk and dropping it: a dropped list keeps a recipe and is recomputed from its parents, which are alive higher up the search, when it is next needed. Costs are measured during the run. The extension items carry only their last item; the prefix is passed down the search, so no itemset is copied per extension. EUCS [4] is stored as a dense triangular matrix of saturating 32-bit cells when it fits (a saturated cell never prunes) and otherwise in hashed partitions; it is never truncated, because a truncated EUCS silently drops valid itemsets.

Projection engine. The database and every projection live in pool pages when they do not fit; EFIM's transaction merging works on the same pages. EFIM-Closed keeps a spilled projection and the projection of its child resident at the same time, hence two per worker.

Tree engine. A spilled prefix tree is mined one node-link step at a time, and each step can be a page load: UP-Growth+ on retail at 32 MB read 207 GB of pages for a 26 MB tree. When the tree's size bound (one node per item occurrence) exceeds the free budget, the miners switch to partition projection: each transaction is written to the partition of its last item; items are processed from the bottom of the order up, each with a small tree built in RAM and mined with

TABLE I
THE OPERATOR MATRIX: EACH ENGINE'S STATE AND THE MOVE APPLIED TO IT WHEN THE BUDGET IS REACHED. S SPILL, R RECOMPUTE, P PARTITION, X RELAX. THE LAST COLUMN IS THE RESIDENT CORE THAT ADMISSION CONTROL REQUIRES.

Engine (algorithms)	Input DB	Base structures	Derived search state	Pruning aids	Candidates	Resident core
Utility lists (HUI-Miner, FHM, FHM+, HUP-Miner, mHUIMiner, TKO, REPT, HAUI-Miner, HUIM-MMU, IncFHM, SHUIM)	S filtered DB spool	S 1-itemset lists by external sort, multi-pass merge	S list chunks or R dropped list rebuilt from parents (cost model)	P EUCS built in partitions; X EUCS disabled, never truncated	none	list headers; one streaming join per worker; top-k heap
Projection (EFIM, EFIM-Closed)	S paged database	item renaming (resident)	S projections as pool pages; P first-level tasks in batches	sub-tree and local utility arrays (resident)	none	one projection per worker (two for EFIM-Closed)
Trees (IHUP, UP-Growth, UP-Growth+, HUP-Tree, HUI-Trie, TKU)	S database spool	P one small tree per item instead of the global tree	P conditional trees inside the item's partition	node bounds per partition (valid, at least as tight)	S spooled; P verified in bounded rounds	TWU map; two node pages; spool buffers
Two-phase (Two-Phase)	S filtered DB spool	S 1-itemset lists	S list chunks	TWU only	S on disk; P verified in batches	list headers; one join per worker
Heuristics (HUIM-GA, HUIM-BPSO, MHUI-ACO)	S DB segments	S streamed per selected item	population (resident)	X reported set becomes a Bloom filter (approximate by design)	none	per-item index; one chunk per item

the unchanged per-item code; then each row moves to the partition of its new last item. All disk access is sequential. Candidate itemsets are spooled to disk and verified, then cleared, whenever the spool reaches a bounded size, instead of all at the end.

Resulting bound. If the ledger stays within B minus the margin, and the margin covers what the ledger cannot see, the RSS stays within B. The first condition holds by construction: every growing structure is reserved before it is allocated. The second is empirical. We calibrated the margin on chainstore (RSS exceeds the ledger peak by about 6.5 MB at 24–64 MB budgets and about 30 MB at 1 GB) and check it on every run (Section VI-C).

VI. EVALUATION

A. Setup

Hardware. Raspberry Pi 4 Model B, 4 GB RAM, quad-core Cortex-A72 at 1.8 GHz, 64 GB SD card, Debian 13, Linux 6.18. CPU frequency stayed at 1.8 GHz with no throttling in any run. Speed comparisons at one thread each, and thread scaling beyond four threads, were run on a laptop (AMD Ryzen 7 7435HS, 8 cores and 16 threads, 23 GB RAM; Section VI-G).

Software. Air-HUIM built with Rust 1.99 in release mode. SPMF 2.67a [8] on OpenJDK 25 with `-Xmx` set to the budget. Both read the same files (checksums recorded per run) with the same thresholds.

Datasets. Table II lists the 14 standard benchmark datasets and LGSR, a simulated but realistic retail dataset from dunnhumby [20], converted to 47.1 M baskets (3.4 GB in SPMF format).

Protocol. A harness runs one job at a time, samples RSS every 100 ms, and records status, wall time, peak RSS, the binary's hash and the dataset's checksum. Runs are capped at 10 min (LGSR: 4 h). Exactness is checked against a reference output computed by Air-HUIM FHM at a large budget; every disagreement between implementations is settled by a full scan of the dataset (Section VII). A run is *out of memory* when the JVM reports it, or when the kernel limit or the watchdog kills it.

TABLE II
DATASETS AND THRESHOLDS (% OF TOTAL UTILITY).

Dataset	Transactions	Items	Avg. length	min_util (%)
foodmart	4,141	1,559	4.4	0.025
chess	3,196	75	37.0	25
mushroom	8,124	119	23.0	5
ecommerce	14,975	3,468	11.6	0.2
pumsb	49,046	2,113	74.0	23
liquor	52,131	4,026	7.9	0.005
BMS	59,602	497	2.5	2.1
connect	67,557	129	43.0	32
retail	88,162	16,470	10.3	0.01
fruithut	181,970	1,265	3.6	0.01
accidents	340,183	468	33.8	12
kosarak	990,002	41,270	8.1	1
chainstore	1,112,949	46,086	7.2	0.1
chicago	2,662,309	35	1.8	0.005
LGSR	47,098,416	4,997	6.5	0.2

B. Exactness

All 905 Air-HUIM runs with a reference output matched it exactly, at every budget and thread count. Top-k and REPT runs have no fixed reference; a full scan confirmed every utility in the top-k outputs on every dataset. The three metaheuristics are approximate by design and excluded.

C. Budget adherence

Of 1,503 Air-HUIM runs, 1,489 peaked at or below the budget. Thirteen exceeded it by 0.1–3.2 MB, all at 24–64 MB;

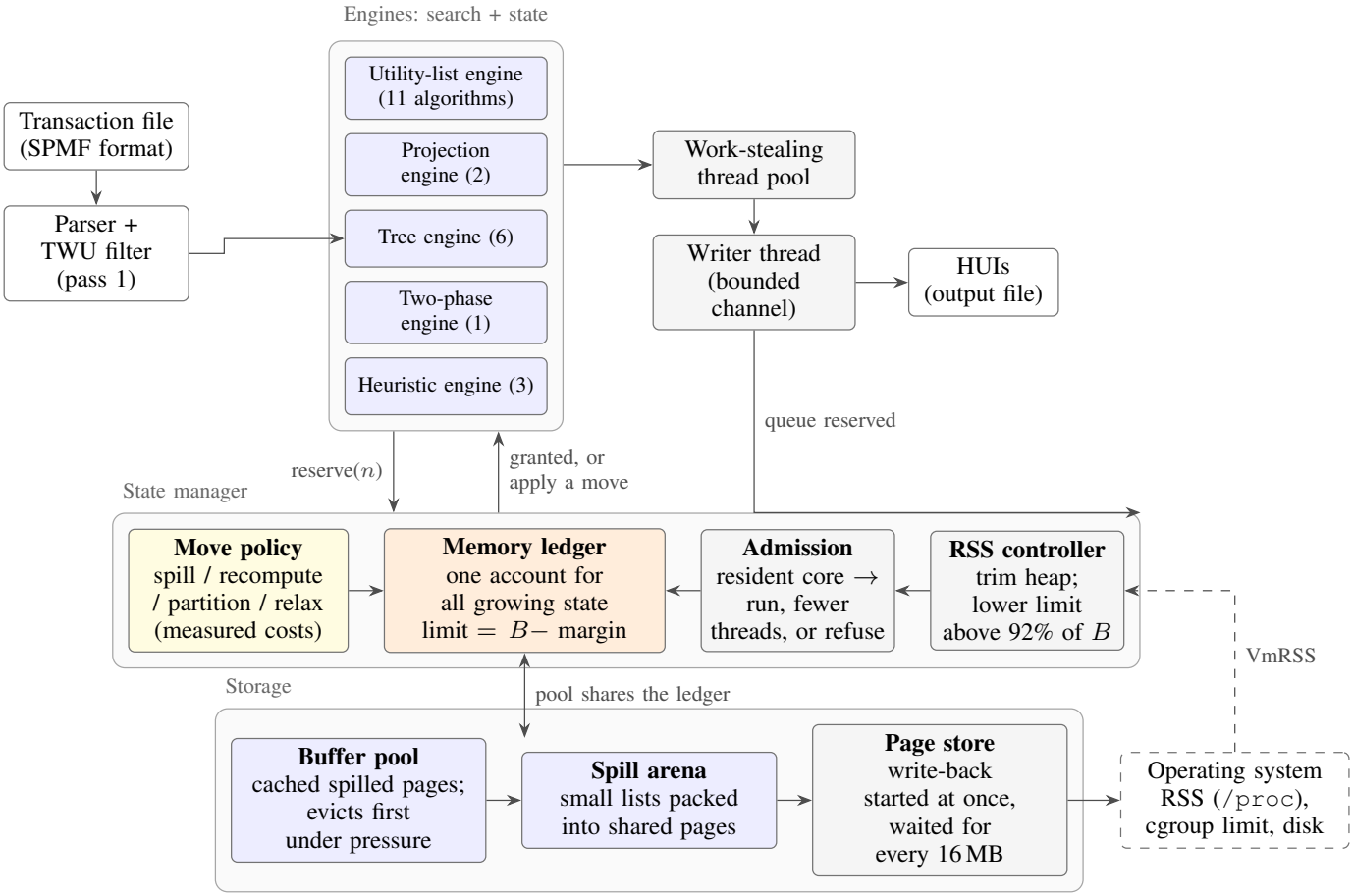


Fig. 3. Air-HUIM's architecture. Engines run the search and hold its state, but every byte of state that grows with the data is reserved from the state manager first; a refused reservation makes the engine apply one of the four moves. The buffer pool shares the same ledger, admission control sizes the resident core before mining, and the RSS controller corrects the ledger with the memory the process really uses.

TABLE III
RUNS KILLED UNDER HARD MEMORY LIMITS (RASPBERRY PI).

Limit	Budgets	Air-HUIM killed	SPMF killed
cgroup `MemoryMax`	64–256 MB	0 / 90	72 / 90
Watchdog (RSS > budget)	64–256 MB	0 / 32	32 / 32

the fourteenth, EFIM-Closed on LGSR at 2 GB, peaked at 2,075 MB (+1.3%). Fig. 4 compares peak memory with the budget for every comparison run: SPMF's JVM exceeds a 64 MB heap budget in every run, by up to 2.6×. Table III shows both tools under two hard limits: a kernel-enforced cgroup memory limit, and a watchdog that kills any process whose RSS exceeds the budget.

D. Air-HUIM versus SPMF at the same budget

For every algorithm both libraries implement, we ran both on each dataset at 64 MB, 256 MB and 1 GB: 196 pairs per budget. Fig. 5 counts the outcomes.

At 64 MB, only Air-HUIM finished in 68 pairs and only SPMF in none. Where neither finished, Air-HUIM hit the time cap without running out of memory; SPMF ran out of memory in 56 of those 60 pairs at 64 MB. In total SPMF ran out of memory in 116, 79 and 34 pairs at 64 MB, 256 MB and 1 GB.



Fig. 4. Peak RSS divided by the budget, every Air-HUIM and SPMF run of the comparison (Pi, log scale). Crosses are runs that ran out of memory. SPMF's heap limit does not bound the process: the JVM's own memory comes on top.

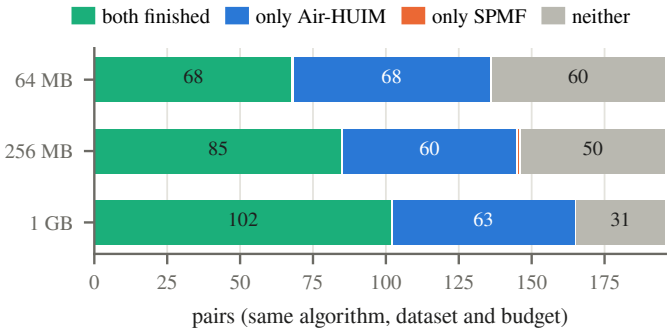


Fig. 5. Outcome of the 196 Air-HUIM/SPMF pairs at each budget on the Pi (same algorithm, dataset and budget; 10 min cap). Only SPMF finished in one pair, at 256 MB.

E. Graceful degradation

Fig. 6 shows Air-HUIM's run time as the budget shrinks from 2 GB to 32 MB. Above the size of the working set, time is flat. Below it, time rises smoothly as more state is spilled or recomputed; there is no cliff where a run fails, and the output is identical at every budget. Two-Phase does not finish on accidents, kosarak and retail at any budget: its candidate set is too large at these thresholds, independent of memory.

F. 47 million baskets on a Raspberry Pi

Fig. 7 shows LGSR on the Pi at 1 and 2 GB, with 4 threads for Air-HUIM. SPMF ran out of memory in all 8 runs, after 1–13 minutes. Air-HUIM finished 7 of 8 with the exact 52 HUIs; EFIM-Closed at 1 GB reached the 4-hour cap without exceeding its budget. Air-HUIM's peaks were 894–1,982 MB for FHM, EFIM and EFIM-Closed, all within budget except EFIM-Closed at 2 GB (2,075 MB).

UP-Growth+ needs about 100 MB at either budget: partition projection keeps only one item's tree in memory at a time.

G. Speed at one thread each

SPMF is single-threaded, so we compare one thread against one thread on a laptop, repeating each of the 102 pairs that both libraries finished at 1 GB on the Pi five times and reporting the median (Fig. 8). Wall time includes process start-up, which costs SPMF about 0.5 s for the JVM. Air-HUIM is faster on 63 of the 102 pairs, with a median speed-up of 1.37×. Counting only the 74 pairs where both tools take at least a second, the two are level: the median is 1.07×, and Air-HUIM is faster on 38.

The spread is wide. EFIM is 17–27× faster on connect, pumsb and liquor, and TKO is 32× faster on retail. In the other direction, EFIM-Closed and TKU on mushroom are 17× and 11× slower, and the utility-list miners on the sparse retail, liquor and fruithut sets are about 3× slower. Budget safety is therefore not free in every case. We have not yet profiled these slow cases.

H. Thread scaling

Fig. 9 shows Air-HUIM on the Pi with 1, 2 and 4 threads at 1 GB. Speed-ups from 1 to 4 threads range from 1.1× to 3.7× (median 1.8×). Short runs (EFIM on chainstore and kosarak, under 12 s) are dominated by reading the input, which is sequential. FHM and HUI-Miner on accidents are slower at 4 threads than at 2.

On the laptop we ran the three large datasets with 1 to 16 threads at 8 GB (Fig. 10); the 42 of 45 runs that finished within the 1 h cap were all exact. HUI-Miner scales best: on LGSR it goes from 37 min at one thread to 6 min at 16 (6.2×), and on chainstore×40 and H&M it finishes only with two and four or more threads, respectively. FHM gains 1.5–1.9× from 1 to 16 threads. EFIM spends most of its 10 s to 80 s reading the input and does not speed up; on LGSR it is 5% slower at 16 threads than at one.

TABLE IV
CORRECTNESS PROBLEMS IN SPMF 2.67A.

Algorithm	Problem	Effect
FHM, FHM+	item comparator casts the 64-bit TWU difference to a 32-bit `int`	once TWUs differ by 2^{31}, the order is inconsistent; on LGSR 20,134 of 1.52 M item pairs are misordered and one true HUI is missed
mHUIMiner	item labels in the output are corrupted	correct count, wrong itemsets on 10 of 12 datasets (retail: 16,308 of 22,479)
TKU	returns fewer than k itemsets	on chicago with $k = 100$ it returns 4; SPMF's own TKO returns 100

VII. Correctness Problems Found in SPMF

Every itemset on which SPMF and Air-HUIM disagreed was settled by computing its exact utility in a plain scan of the dataset, independent of both libraries. Air-HUIM's output was exact in every comparison. SPMF showed three problems (Table IV).

The FHM problem was confirmed in the bytecode (`lsub` followed by `l2i`); it did not appear on the 14 benchmark datasets, but appears on retail data with billions in total utility. The TKU problem reproduces with a 1 GB and a 4 GB heap, so it is not a memory effect.

VIII. Limitations

The budget is not yet a strict ceiling: 14 of 1,503 runs exceeded it, by at most 3.2 MB at small budgets and by 1.3% for EFIM-Closed on LGSR; we have not yet found the allocation the ledger misses. Some algorithms are slow at small budgets for reasons unrelated to memory (TWU-only bounds give too many candidates for IHUP, HUI-Trie and HUP-Tree on retail and kosarak). The Pi experiments ran each configuration once; the speed comparison uses five repeats. LGSR is simulated retail data.

IX. Conclusion

Air-HUIM shows that the standard HUIM algorithms can run exactly under a hard memory budget on a 4 GB single-board computer, including on 47 million baskets, where the reference implementation fails. The method is uniform across

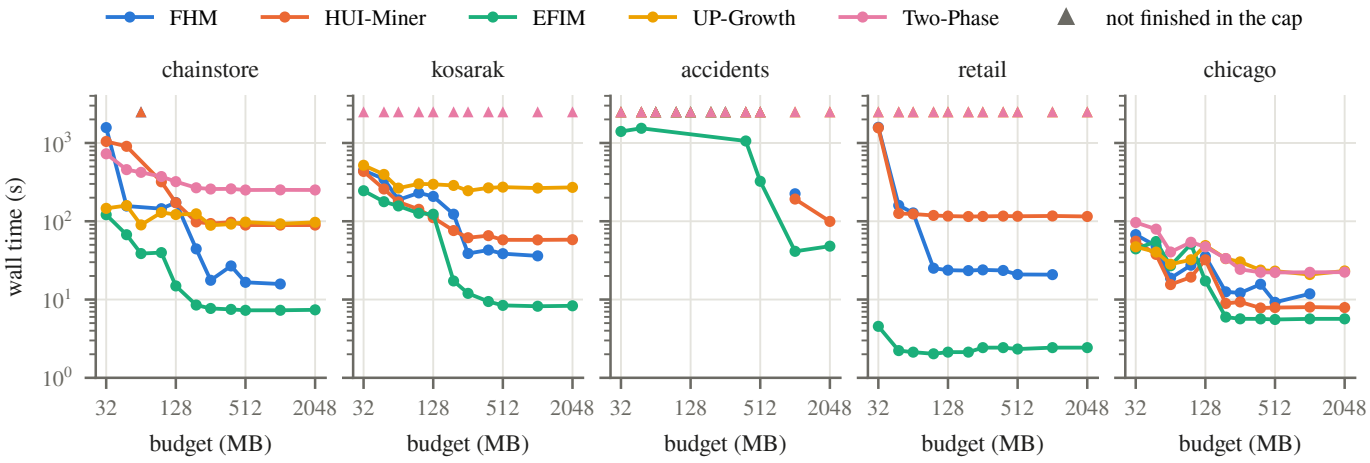


Fig. 6. Air-HUIM wall time against budget on the Pi (4 threads, log–log). Lines join finished runs; triangles mark budgets where a run reached its time cap (10 or 30 min). Output is identical at every budget.

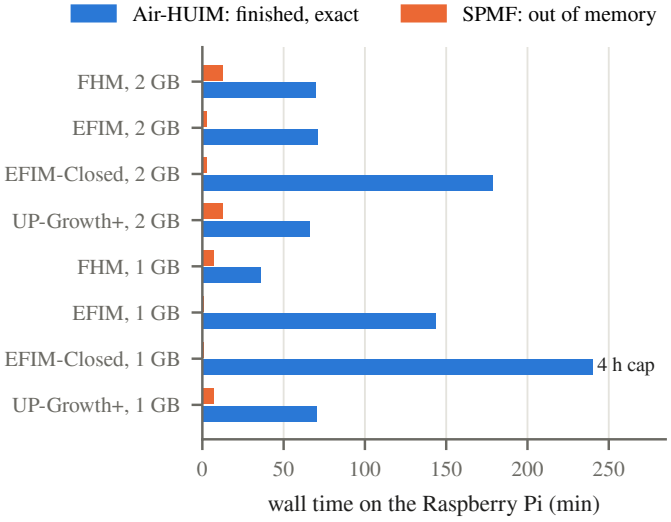


Fig. 7. LGSR (47.1 M baskets, min_util 0.2%) on the 4 GB Raspberry Pi: time until SPMF ran out of memory, and time until Air-HUIM finished.

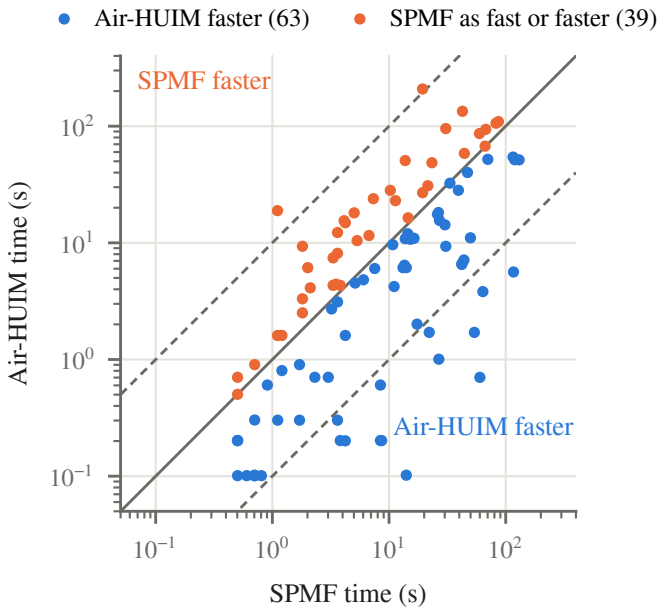


Fig. 8. One thread against one thread: median of five runs for each of the 102 pairs both libraries finished, 1 GB budget, laptop. Solid line: equal time; dashed lines: 10× either way.

23 algorithms: one ledger, spill or recompute or partition, and clean refusal below a computed minimum. The library, benchmark harness and all run records are available at https://github.com/CyberKnight-cmd/pocket-data-mining and archived as release 1.0.0 at https://doi.org/10.5281/zenodo.24039625.

References

[1] Y. Liu, W. keng Liao, and A. Choudhary, "A two-phase algorithm for fast discovery of high utility itemsets," in *Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining (PAKDD)*, ser. LNCS, vol. 3518, 2005, pp. 689–695.

[2] W. Gan, J. C.-W. Lin, P. Fournier-Viger, H.-C. Chao, V. S. Tseng, and P. S. Yu, "A survey of utility-oriented pattern mining," *IEEE Trans. Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1306–1327, 2021.

[3] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *Proc. ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2012, pp. 55–64.

[4] P. Fournier-Viger, C.-W. Wu, S. Zida, and V. S. Tseng, "FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning," in *Proc. Int. Symp. Methodologies for Intelligent Systems (ISMIS)*, ser. LNCS, vol. 8502, 2014, pp. 83–92.

[5] S. Zida, P. Fournier-Viger, J. C.-W. Lin, C.-W. Wu, and V. S. Tseng, "EFIM: a fast and memory efficient algorithm for high-utility itemset mining," *Knowledge and Information Systems*, vol. 51, no. 2, pp. 595–625, 2017.

[6] V. S. Tseng, B.-E. Shie, C.-W. Wu, and P. S. Yu, "Efficient algorithms for mining high utility itemsets from transactional databases," *IEEE Trans. Knowledge and Data Engineering*, vol. 25, no. 8, pp. 1772–1786, 2013.

[7] C. Zhang, G. Almpanidis, W. Wang, and C. Liu, "An empirical evaluation of high utility itemset mining algorithms," *Expert Systems with Applications*, vol. 101, pp. 91–115, 2018.

[8] P. Fournier-Viger, J. C.-W. Lin, A. Gomariz, T. Gueniche, A. Soltani, Z. Deng, and H. T. Lam, "The SPMF open-source data mining library

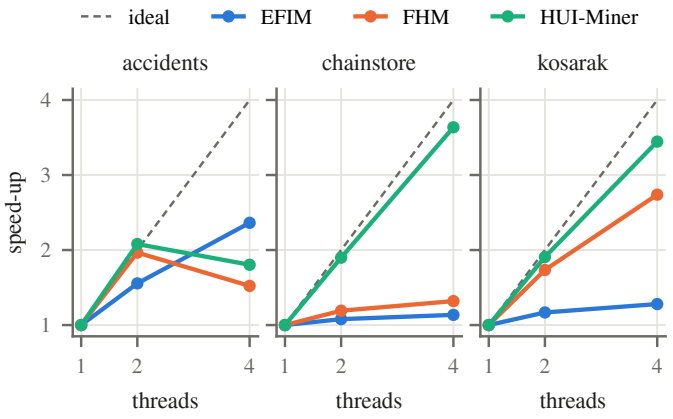


Fig. 9. Speed-up over one thread on the 4-core Pi (1 GB budget). HUI-Miner scales best; EFIM's short runs are dominated by sequential input reading.

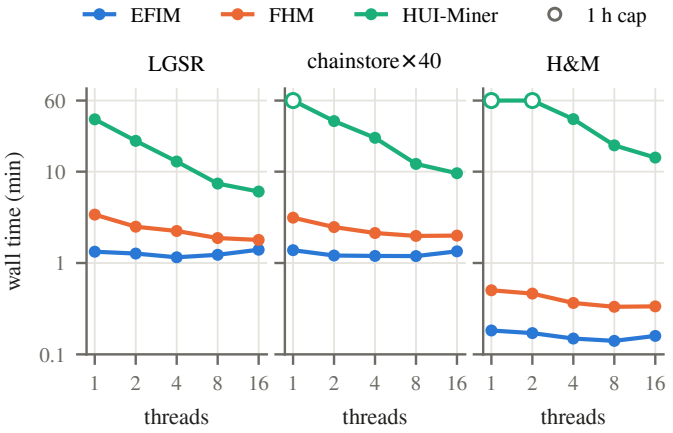


Fig. 10. Wall time against threads on the 16-thread laptop (8 GB budget, log scales). Hollow circles: stopped at the 1 h cap.

version 2," in *Proc. ECML PKDD (Part III)*, ser. LNCS, vol. 9853, 2016, pp. 36–40.

[9] G. Kimura, Y. Hayamizu, R. U. Kiran, M. Kitsuregawa, and K. Goda, "DPHIM: Efficient parallel mining of high-utility itemsets on multicore processors and its evaluation," *IEEE Trans. Knowledge and Data Engineering*, 2026.

[10] T. Sreepada, T. Ozawa, G. Kimura, R. U. Kiran, and K. Goda, "Adaptive GPU compute resource allocation for efficient high-utility itemset mining," in *Lecture Notes in Computer Science*, 2026.

[11] A. Y. Peng, Y. S. Koh, and P. Riddle, "mHUIMiner: A fast high utility itemset mining algorithm for sparse datasets," in *Proc. PAKDD*, ser. LNCS, vol. 10235, 2017, pp. 196–207.

[12] V. S. Tseng, C.-W. Wu, P. Fournier-Viger, and P. S. Yu, "Efficient algorithms for mining top-k high utility itemsets," *IEEE Trans. Knowledge and Data Engineering*, vol. 28, no. 1, pp. 54–67, 2016.

[13] R. U. Kiran, P. Veena, M. Toyoda, and M. Kitsuregawa, "PAMI: An open-source python library for pattern mining," *Journal of Machine Learning Research*, vol. 25, 2024.

[14] G. Kimura, Y. Hayamizu, R. U. Kiran, M. Kitsuregawa, and K. Goda, "Efficient parallel mining of high-utility itemsets on multicore processors," in *Proc. IEEE Int. Conf. Data Engineering (ICDE)*, 2023.

[15] A. Erwin, R. P. Gopalan, and N. R. Achuthan, "Efficient mining of high utility itemsets from large datasets," in *Proc. PAKDD*, ser. LNCS, vol. 5012, 2008, pp. 554–561.

[16] D. T. Vu and H. D. Nguyen, "Mining high utility itemsets in massive transactional datasets," *Acta Cybernetica*, vol. 20, no. 2, 2011.

[17] X. Han, X. Liu, J. Li, and H. Gao, "Efficient top-k high utility itemset mining on massive data," *Information Sciences*, vol. 557, pp. 382–406, 2021.

[18] G. Buehrer, S. Parthasarathy, and A. Ghoting, "Out-of-core frequent pattern mining on a commodity PC," in *Proc. ACM SIGKDD*, 2006, pp. 86–95.

[19] J. Saffran, G. Garcia, M. A. Souza, P. H. Penna, M. Castro, L. F. W. Góes, and H. C. Freitas, "A low-cost energy-efficient Raspberry Pi cluster for data mining algorithms," in *Euro-Par 2016 Workshops*, ser. LNCS, vol. 10104, 2017, pp. 788–799.

[20] dunnhumby, "Let's get sort-of-real (dummy data)," https://www.dunnhumby.com/source-files/, 2026.